

Core Idea. We adopt a two-stage prompt optimization strategy that combines textual and numeric gradients to leverage their complementary strengths. Textual gradients enable large improvements in the early stages by making high-level semantic revisions to the prompt; however, as optimization approaches a near-optimal region, these updates tend to become unstable and may fluctuate. In contrast, numeric gradient-based optimization progresses more slowly but provides fine-grained guidance by identifying which specific tokens are most important to update. Therefore, we first apply textual gradient updates to quickly reach a strong initial solution, and then refine it using numeric gradients to achieve more stable and precise improvements near the optimum.

*Justification of Line 19 in Algorithm 3: *Why we prepend the original prompt as a prefix during autoregression:* We append the original prompt prefix along with a paraphrasing instruction to preserve task context during autoregressive generation. Using only the prefix of the prompt provides insufficient information about the task and ignores the right-side context of the prompt, leading the LLM to generate unrelated or inconsistent tokens. Including this context ensures that candidate tokens remain aligned with both the task and the overall prompt structure.

Algorithm 1 Prompt Optimization

Input: Initial prompt P , training set $\mathcal{D}_{\text{train}}$, validation set $\mathcal{D}_{\text{val}}$

Output: Optimized prompt P^*

```

1:  $\mathcal{P}_{\text{text}} \leftarrow \{P\}$  ▷ Textual population
2: for  $t = 1$  to  $T$  do
3:   Sample  $P_t \sim \mathcal{P}_{\text{text}}$ 
4:    $P' \leftarrow \text{TextualGradient}(P_t, \mathcal{D}_{\text{train}})$ 
5:   Add  $P'$  to  $\mathcal{P}_{\text{text}}$  with validation score on  $\mathcal{D}_{\text{val}}$ 
6: end for
7:  $P^* \leftarrow \arg \max_{P \in \mathcal{P}_{\text{text}}} \text{Score}(P, \mathcal{D}_{\text{val}})$ 
8:  $\mathcal{P}_{\text{num}} \leftarrow \{P^*\}$  ▷ Numeric population
9: for  $q = 1$  to  $Q$  do
10:  Sample  $P_q \sim \mathcal{P}_{\text{num}}$ 
11:   $\mathcal{P}' \leftarrow \text{NumericGradient}(P_q, \mathcal{D}_{\text{train}})$ 
12:  for all  $p \in \mathcal{P}'$  do
13:    Add  $p$  to  $\mathcal{P}_{\text{num}}$  with validation score on  $\mathcal{D}_{\text{val}}$ 
14:  end for
15: end for
16:  $P^* \leftarrow \arg \max_{P \in \mathcal{P}_{\text{num}}} \text{Score}(P, \mathcal{D}_{\text{val}})$ 
17: return  $P^*$ 

```

Algorithm 2 Textual Gradient Update

Input: Prompt P_x , training set $\mathcal{D}_{\text{train}}$

Output: Updated prompt P'_x

```

1: Sample a mini-batch  $\mathcal{B} \subset \mathcal{D}_{\text{train}}$ 
2: Use an LLM to infer predictions for  $\mathcal{B}$  under  $P_x$ 
3: Construct a feedback set  $\mathcal{D}_{\text{feedback}}$  with three correct and three incorrect predictions
4: Use an LLM to analyze  $\mathcal{D}_{\text{feedback}}$  and produce feedback  $R$ 
5: Revise  $P_x$  using  $R$  to obtain  $P'_x$ 
6: return  $P'_x$ 

```

Algorithm 3 Numeric Gradient Update

Input: Prompt P_x , training set $\mathcal{D}_{\text{train}}$, method m , feedback sample size N , number of prompt to generate M , number of prompts to keep K , candidate size E

Output: Top- K updated prompts $\mathcal{P}'$

```
1: Sample  $N$  examples from  $\mathcal{D}_{\text{train}}$  to form  $\mathcal{D}_{\text{feedback}}$ 
2: Convert  $\mathcal{D}_{\text{feedback}}$  into binary support–query pairs
3: Run inference on  $\mathcal{D}_{\text{feedback}}$  using  $P_x$  and obtain predictions  $R$ 
4: Partition  $R$  into confusion buckets  $\{\text{TP}, \text{TN}, \text{FP}, \text{FN}\}$ 
5: Construct a balanced subset  $\mathcal{D}_{\text{grad}}$  by sampling equally from each bucket
6: Compute token-level gradients of  $P_x$  on  $\mathcal{D}_{\text{grad}}$ 
7: Select the top three gradient regions and store them in  $\mathcal{R}$ 
8: Initialize an empty candidate map  $\mathcal{C}$ 
9: if  $m = \text{GENERATE\_CANDIDATES\_BY\_PROMPTING}$  then
10:   for all  $r \in \mathcal{R}$  do
11:      $\tilde{P} =$  updated prompt  $P_x$  where region  $r$  is marked using a <target> span
12:      $P_{\text{meta}} =$  meta-prompt requesting  $E$  replacements (i.e. candidates) of region  $r$  in  $\tilde{P}$ 
13:     Generate candidate tokens  $C_r$  using the LLM with  $P_{\text{meta}}$ 
14:     Store  $C_r$  in  $\mathcal{C}$  for region  $r$ 
15:   end for
16: else if  $m = \text{GENERATE\_CANDIDATES\_BY\_AUTOREGRESSION}$  then
17:   for all  $r \in \mathcal{R}$  do
18:     Let  $i$  be the index of region  $r$  in  $P_x$ 
19:     Construct  $\tilde{P} = P_x + \text{paraphrase instruction} + P_x[0 : (i - 1)]$ 
20:     Candidate tokens,  $C_r = \arg \text{topK}_{t \in V} \text{LLM}(t \mid \tilde{P})$ 
21:     Store  $C_r$  in  $\mathcal{C}$  for region  $r$ 
22:   end for
23: end if
24: Construct a meta-prompt requesting  $M$  combinations of candidates from  $\mathcal{C}$ 
25: Generate  $M$  candidate combinations  $\mathcal{C}_{\text{comb}}$  by running the meta-prompt
26: Initialize an empty prompt set  $\mathcal{P}_{\text{cand}}$ 
27: for all  $c \in \mathcal{C}_{\text{comb}}$  do
28:   Construct a meta-prompt that replaces each region  $r$  in  $P_x$  with its selected candidate  $c_r$ 
29:   Generate a new prompt  $P_{\text{new}}$  running the meta-prompt
30:   Add  $P_{\text{new}}$  to  $\mathcal{P}_{\text{cand}}$ 
31: end for
32: Initialize an empty score map  $\mathcal{S}$ 
33: for all  $P \in \mathcal{P}_{\text{cand}}$  do
34:   Compute  $s = \text{Loss}_{\text{cross\_entropy}}(P, \mathcal{D}_{\text{grad}}) + \lambda \text{Perplexity}(P)$ 
35:   Store score  $s$  in  $\mathcal{S}$  for prompt  $P$ 
36: end for
37: Rank prompts in  $\mathcal{P}_{\text{cand}}$  according to  $\mathcal{S}$ 
38: Let  $\mathcal{P}'$  be the top- $K$  prompts of  $\mathcal{P}_{\text{cand}}$ 
39: return  $\mathcal{P}'$ 
```
